Landform Mesh Tiling Server SETUP

Contents

AWS Setup

The AWS services used are

- Elastic Compute Cloud EC2 (https://aws.amazon.com/ec2)
- Simple Cloud Storage Service S3 (https://aws.amazon.com/free/storage)
- DynamoDB (https://aws.amazon.com/dynamodb)
- Simple Queue Service SQS (https://aws.amazon.com/sqs)
- Elastic Beanstalk EB (https://aws.amazon.com/elasticbeanstalk)
- Identity & Access Management IAM (https://aws.amazon.com/iam)
- Managed Cloud DNS Route 53 (https://aws.amazon.com/route53) (optional)
- Certificate Manager (https://aws.amazon.com/certificate-manager)

A full deployment includes

1. a master server running on an EC2 instance managed with Elastic Beanstalk
2. a group of workers running on one or more EC2 instances managed as an EC2 Auto Scaling Group.

The master server includes

- a web interface that can be securely accessed by end users
- a REST API that can be used to securely integrate with other systems
- a backend task which orchestrates tiling workflows.

The workers perform tasks defined by the master and are not externally accessible.

Each deployment on AWS is configured with a unique venue name used to partition the DynamoDB, S3, and SQS entries specific to that venue.

1. S3 Bucket Setup

Input and output datasets are stored an AWS S3 bucket. The bucket needs to be fully accessible by the account in which the server components (master and workers) are deployed. Typically this is ensured by using the same account to create the S3 bucket as for deploying the server components.

The bucket typically also needs external `https` access so that results can be accessed for viewing and downstream use. The instructions below show how to set this up limited to clients with JPL IP addresses.

1. http://goto.jpl.nasa.gov/awsconsole
2. Log in with the same AWS account you use to deploy the server components
3. Select region `us-west-1` (North California)
4. Services -> Storage -> S3
5. Create Bucket
    1. enter a bucket name - note it must be globally unique across all S3 bucket names
    2. Create
6. Select bucket in list
    1. Permissions -> Bucket Policy
        1. paste the following, replacing `BUCKET_NAME` with your bucket name

```
{
  "Version": "2012-10-17",
  "Id": "S3PolicyId1",
  "Statement": [
    {
      "Sid": "allow readonly from JPL",
      "Effect": "Allow",
      "Principal": {
          "AWS": "*"
      },
      "Action": "s3:GetObject",
```

```
            "Resource": "arn:aws:s3:::BUCKET_NAME/*",
            "Condition": {
              "IpAddress": {
                "aws:SourceIp": [
                  "128.149.0.0/16",
                  "137.78.0.0/16",
                  "137.79.0.0/16",
                  "137.228.0.0/16"
                ]
              }
            }
          }
        ]
      }
```

2. Save

2. Properties -> Static website hosting
    1. Use this bucket to host a website
    2. Index document: `index.html`
    3. Save

2. Create Elastic Beanstalk Environment for Master Server

This step creates the Elastic Beanstalk application and environment into which the master server will be deployed.

1. Log in to the AWS web console with an AWS account and region that can access the S3 bucket you setup above.
2. Services -> Compute -> Elastic Beanstalk
3. Create application and assign it a unique name
4. Create new web sever environment
    1. assign a unique environment name
    2. Platform: `docker`
    3. Configure more options
        1. Modify instances
            1. Instance type: `t2.medium`
            2. Instance security groups: default creates a security group that can only talk to the load balancer which is what we want
    4. Modify Capacity -> Load balanced, min = 1, max = 1
    5. Modify security -> IAM instance profile: typically set this to match your AWS account
    6. Modify network
        1. Visibility: Public
        2. Load balancer subnets:
           `us-west-1c subnet-148d7971 172.31.16.0/20`
        3. Instance subnets: same as load balancer
5. Create Environment - it takes a few minutes

6. Adjust elastic load balancer settings.
    1. Log in to the AWS web console with the same AWS account and region as above.
    2. Services -> Compute -> EC2
    3. Load Balancing -> Load Balancers
    4. Select the load balancer corresponding to the elastic beanstalk environment - on the "Instances" tab you should see an instance with the same name as the elastic beanstalk environment
    5. On the "Description" tab for the load balancer, click "Edit idle timeout" and set it to 1800 seconds
    6. On the "Listeners" tab, click "Edit", and then change cipher.
        1. Choose "Custom Security Policy"
        2. under SSL Protocols make sure TLSv1 is unchecked (TLSv1.1 and greater are OK)
        3. under SSL Ciphers make sure only ones from the following NASA approved list https://jplsoc2.jpl.nasa.gov/jplsoc/compliance/ciphers/supported_ciphers.txt are checked (the `TLS_` prefix may be missing)
            - `TLS_DHE_RSA_WITH_AES_128_GCM_SHA256`
            - `TLS_DHE_RSA_WITH_AES_256_GCM_SHA384`
            - `TLS_DHE_DSS_WITH_AES_128_CBC_SHA`
            - `TLS_DHE_DSS_WITH_AES_256_CBC_SHA`
            - `TLS_DHE_DSS_WITH_AES_128_GCM_SHA256`
            - `TLS_DHE_DSS_WITH_AES_256_CBC_SHA256`
            - `TLS_DHE_DSS_WITH_AES_256_GCM_SHA384`
            - `TLS_DHE_DSS_WITH_AES_128_CBC_SHA256`
            - `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`
            - `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`
            - `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256`
            - `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384`
            - `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`
            - `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`
            - `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`
            - `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`
            - `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384`
            - `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256`
            - `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`
            - `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`
        4. save the changes
        5. once you complete the DNS setup steps below, for deployments at JPL you can check that you got them all by going to http://jplsoc2.jpl.nasa.gov/ciphers/ciphers_check.cfm and verifying that no ciphers are shown in red for the DNS name of your depolyment.

3. Configure DNS

This step is optional. It configures a public DNS entry to redirect to the Elastic Beanstalk environment configured above. These instructions use the Amazon Route 53 DNS service, but any DNS provider that supports CNAME records should work.

If you forgo this step you can still access the server at a URL like `https://EBENV.AWSREGION.elasticbeanstalk.com` where `EBENV` is the Elastic Beanstalk environment name and AWSREGION is the AWS region containing it.

1. Log in to the AWS web console, does not need to be the same AWS account and region as above.
2. Services -> Networking -> Route 53
3. select hosted zone
4. Create record set
    1. Name: DNS name you want end-users to use to access the server
    2. Type: `CNAME`
    3. TTL: `60`
    4. Value: `EBENV.AWSREGION.elasticbeanstalk.com` - this must match the Elastic Beanstalk environment created above
    5. Routing policy: `simple`
    6. Create

4. Configure HTTPS

This step uses the JPL and AWS certificate managers to generate and deploy cryptographic certificates to enable secure HTTPS connection to the server.

The overall flow is

1. generate an RSA private key
2. use the key to sign a certificate signing request (CSR)
3. submit the CSR to the JPL certificate manager to generate an HTTPS certificate (PEM)
4. register the certificate and key to the AWS certificate manager
5. associate the certificate with an Elastic Beanstalk load balancer listener which will proxy HTTPS communication with the Landform master server.

Many steps below require the end-user DNS name for your server which will be noted as `DNS_NAME`.

1. Generate Certificate Signing Request

You will need a command line that includes the `openssl` tool. On Windows one option is Git bash (https://gitforwindows.org).

1. generate an RSA private key: `openssl genrsa > DNS_NAME.key`

2. generate CSR: `openssl req -new -key DNS_NAME.key -out DNS_NAME.csr`
    - Common name: `DNS_NAME`
    - other values will be specific to your deployment

2. Generate Signed Certificate

Now you will submit the CSR to a certificate signing authority to generate a signed certificate. The instructions below can be used within JPL.

1. Login to JPL Certificate manager at https://ssl.jpl.nasa.gov
2. Manage my certificates -> Request a Certificate
    1. LDAP group: select an LDAP group for managing the certificate
    2. Machine name: `DNS_NAME`
    3. Server type: Apache
    4. CSR: paste entire contents of the CSR here
    5. wait a few minutes, you should get an email when the cert is ready
3. Manage my certificates
    1. click on `DNS_NAME`
    2. paste entire contents of certificate to `DNS_NAME.pem`. It shold have a form like this

```
CERTIFICATE:
-----BEGIN CERTIFICATE-----
(cert)
-----END CERTIFICATE-----
INTERMEDIATE CERTIFICATE:
-----BEGIN CERTIFICATE-----
(intermediate cert)
-----END CERTIFICATE-----
ROOT CERTIFICATE:
-----BEGIN CERTIFICATE-----
(root cert)
-----END CERTIFICATE-----
```

3. Install Signed Certificate

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Security -> Certificate Manager -> Import a certificate
    1. Paste

```
-----BEGIN CERTIFICATE-----
(cert)
-----END CERTIFICATE-----
```

    from your PEM file into the certificate body field.
    2. Paste

```
-----BEGIN CERTIFICATE-----
(intermediate cert)
-----END CERTIFICATE-----
```

`-----BEGIN CERTIFICATE-----`
`(root cert)`
`-----END CERTIFICATE-----`
from your PEM file into the certificate chain field.
3. Paste entire contents of your RSA private key file into the certificate private key field.
4. Review and Import -> Import
5. Open accordion for the cert
   - edit name tag to be your Landform master server DNS name
   - Details -> Identifier GUID
   - Make a note of the GUID

3. Services -> Compute -> Elastic Beanstalk
   1. Navigate into the Elastic Beanstalk application and environment you configured above
   2. Configuration -> Modify load balancer
      1. Classic load balancer
      2. Turn off existing listener
      3. Add listener
         - Listener port: `443`
         - Listener protocol: `HTTPS`
         - Instance port: `80`
         - Instance protocol: `HTTP`
         - Select the SSL certificate ID. These may appear as `*.jpl.nasa.gov`, in which case you need to find the GUID matching the cert in the AWS Certificate Manager.
      4. Apply

5. Restrict to JPL IPs

This step is optional but recommended for deployments within JPL. It restricts access to your Landform master server to clients within the JPL IP address space.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Configuration -> Instance settings, note down the EC2 security group
5. Services -> Compute -> EC2 -> Security Groups
   1. Search for the instance security group noted above
   2. Look at the security group referenced in its inbound source. This is the ELB security group.
   3. Search for and select the ELB security group and change its inbound settings to:
      - `HTTPS TCP 443 128.149.0.0/16`

- HTTPS TCP 443 137.78.0.0/16
- HTTPS TCP 443 137.79.0.0/16
- HTTPS TCP 443 137.228.0.0/16

6. Configure Elastic Beanstalk Environment

This step configures the Elastic Beanstalk environment with specifics of your deployment.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Software -> Environment variables
   - `NODE_ENV`: `production` (`integration` for testing)
   - `SESSION_SECRET`: any private string
   - `TOKEN_SECRET`: any private string
   - `SAML_ENTRY_POINT`: for depoloyments within JPL you will need to contact JPL IT to set up SSO for your deployment and typically this field will have a form like `https://SSO_HOST.jpl.nasa.gov/oamfed/idp/initiatesso?providerid=https://DNS_NAME` where `SSO_HOST` is `ssoint` for integration testing and `sso1` for production, and `DNS_NAME` is your server DNS name.
   - `SAML_CERT`: for deployments within JPL typically copy the X509Certificate field from `https://SSO_HOST.jpl.nasa.gov/oamfed/idp/metadata`.
   - `LANDFORM_AWS_REGION`: same region you used to setup the Elastic Beanstalk environment
   - `LANDFORM_AWS_PROFILE`: `default`
   - `LANDFORM_VENUE`: choose a venue name for the deployment; it must match the venue name in the worker configuration below
   - `LANDFORM_S3_URL`: `s3://BUCKET_NAME` where `BUCKET_NAME` is the S3 bucket you setup above
   - `LANDFORM_LDAP_GROUP`: LDAP group for SSO authentication
5. Apply

7. Deploy Server Binaries to Elastic Beanstalk

The following instructions assume you have a `landform-master[-VERSION].zip` release bundle.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Upload and Deploy

5. Choose File: `landform-master[-VERSION].zip`
6. Version Label: `VERSION`
7. Deploy - it takes a few minutes

Once the server is deployed it will be live. To shut it down, either terminate the corresponding Elastic Beanstalk environment or set its autoscale group to max 0 instances. The latter may be more convenient because terminating the environment seems to loose its configuration. One way to reduce the autoscale group to 0 instances is to us the Python `eb` command line tool. Using Python 3:

```
pip install awsebcli
eb scale 0 ENVIRONMENT --profile=PROFILE
```

where `ENVIRONMENT` is the Elastic Beanstalk environment name and `PROFILE` is the AWS profile to use.

8. Deploy Worker Binaries to EC2 Auto Scale Group

The following instructions assume you have a `landform-worker[-VERSION].zip` release bundle and an `ec2userdata.txt` file.

1. Setup Security

These steps only need to be performed once before your first deployment.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Network & Security -> Security Groups (optional - only if you don't already have a security group that enables RDP and you want to have remote access to the EC2 instances in the autoscale group for debugging or maintenance)
    1. Create Security Group
    2. Name: `RDP Only` (recommended, but can be any name)
    3. Description: `RDP Only` (recommended, but can be any name)
    4. Rules
        1. Add Rule
            - Type: `RDP`
            - Source:
              `Custom: 128.149.0.0/16, 137.78.0.0/16, 137.79.0.0/16, 137.228.0.0/16`
              this will restrict RDP access to JPL IP addresses
4. Services -> Compute -> EC2
5. Network & Security -> Key Pairs (optional - only if you want to be able to log in to the EC2 instances in the autoscale group for debugging or maintenance)
    1. you can use a tool like `ssh-keygen -t rsa` to generate a key pair

2. Create Launch Template

These instructions only need to be run once for a new venue or when the venue configuration (`ec2userdata.txt`) changes.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Instances -> Launch Templates
4. Create New Launch Template
   - If you have already created a template for a venue then you can
     - select "Create a new template version"
     - select the previous template in "Launch template name"
     - select the most recent version of the previous template in "Source Template Version"
     - the new template will be pre-populated with values from the old template
     - most likely the only field you'll need to replace is "User Data"
     - after saving the new version
       * select the template in the list
       * Actions -> Set default version
       * select the newest version
       * click set as default version
   - Launch Template Name: select your launch template
   - AMI ID: `ami-0df605282263fb1c9` (Microsoft Windows Server 2016 Base 64-bit)
   - Instance Type: `t2.2xlarge` recommended, other instance types (https://aws.amazon.com/ec2/instance-types) can be chosen for different price (https://aws.amazon.com/ec2/pricing/on-demand) and performance tradeoffs
   - Key Pair: the key pair you selected above
   - Network Type: `classic`
   - Availability Zone: same AWS region that you used to set up the Elastic Beanstalk environment
   - Security Groups: `RDP Only` (or whatever security group you selected above)
   - Tags -> Add Tag
     - Key: `Name` - note this must be capitalized exactly as shown
     - Value: assign a name to will identify the EC2 instances in the group
   - Advanced
     - IAM Instance Profile: typically set this to match your AWS account
     - User Data: cut and paste the entire contents of `ec2userdata.txt`
5. Create Launch Tempate

3. Upload New Worker

These instructions only needs to be run when the Landform worker version changes (`landform-worker[-VERSION].zip`).

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Storage -> S3
3. select your S3 bucket
4. select or create a subfolder matching your venue name
5. select or create subfolder `app`
6. if `landform-worker[-VERSION].zip` exists, delete it
7. upload `landform-worker[-VERSION].zip`
8. right click on `landform-worker[-VERSION].zip` and rename to `landform-worker.zip`
9. you will need to restart all EC2 instances in the autoscale group to pick up the changes; one way to do that is to delete any existing auto scaling group and then re-create following the instructions below.

4. Create Auto Scaling Group

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Auto Scaling -> Auto Scaling Groups -> Create Auto Scaling Group
4. Launch Template
5. select the launch template you created above
6. Next Step
   - Group Name: assign a group name, e.g. same as launch template name
   - Subnet: `172.31.16.0/20 | Default in AWS_REGION`
7. Configure Scaling Policies
   - Use scaling policies to ajust the capacity of the group
   - Scale Between: 1 and 4 instances (recommended, but other parameters can be chosen for different price/performance tradeoffs)
   - Metric type: `Average CPU Utilization`
   - Target value: `60`
   - Instances need: `300` seconds to warm up after scaling
8. Review
9. Create Auto Scaling Group

5. To Remote in to an EC2 Instance in the Autoscale Group

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2

3. Instances -> Instances
4. Right click on an instance in the group
5. if this is your first time connecting
    1. Get Password
    2. choose file and select the *private* key from the key pair you specified when creating the launch template above
    3. Decrypt Password
    4. copy to clipboard
6. Download remote desktop (RDP) file
7. Double click RDP file to open remote desktop
8. Username: `admin`, password as above
9. The server should be located in `C:\tileserver`. You can tail the server log by running the PowerShell command

    ```
    Get-Content c:\tileserver\log\log-tilingserver-startworker*.txt -Wait -Tail 30
    ```

10. The EC2Launch (https://docs.aws.amazon.com/AWSEC2/latest/WindowsGuide/ec2-windows-user-data.html) log for the user data script should be at

    ```
    C:\ProgramData\Amazon\EC2-Windows\Launch\Log\UserdataExecution
    ```

    You may need to show hidden files and folders (https://support.microsoft.com/en-us/help/14201/windows-show-hidden-files) to see `C:\ProgramData`.